3rd Top Down Approach Lexical Analysis

Grammar 1

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

1. Draw the parse trees for the following statements using grammar 1:

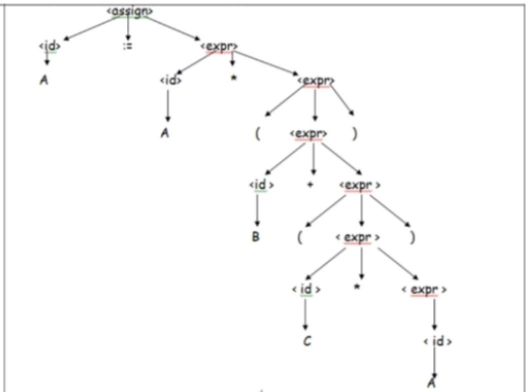
   a. **A:= A * (B + (C * A))**

   b. **B := C * (A * C + B)**

   c. **A : = A * (B + (C))**

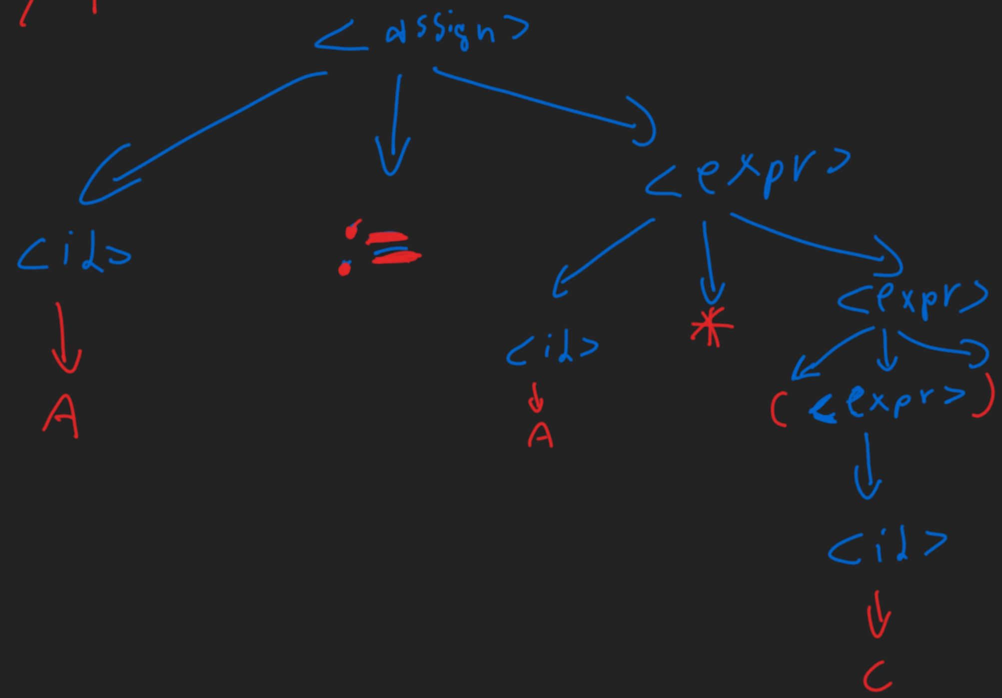
Solution

a.

< assign > → < id > := < expr > → A := < expr > → A := < id > * < expr > → A := A * < expr > → A := A * (< expr >) → A := A * (< id > + < expr >) → A := A * (B + < expr >) → A := A * (B + (< expr >)) → A := A * (B + (< id > * < expr >)) → A := A * (B + (C * < expr>)) → A := A * (B + (C * < id >)) → A := A * (B + (C * A)) ..	

① b)

A : = A * (B + (C))



<assign> → <id> := <expr>
→ A := <expr>
→ A := <id> * <expr>
→ A := A * <expr>
→ A := A * (<expr>)
→ A := A * (<id> + <expr>)
→ A := A * (B + (<expr>))
→ A := A * (B + (<id>))
→ A := A * (B + (C))

Youssef Eslam
231000070
#Assignment 9

Grammar 2

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>

2. Draw the parse trees for the following statements using grammar 2:

a. **A := (A + B) * C**

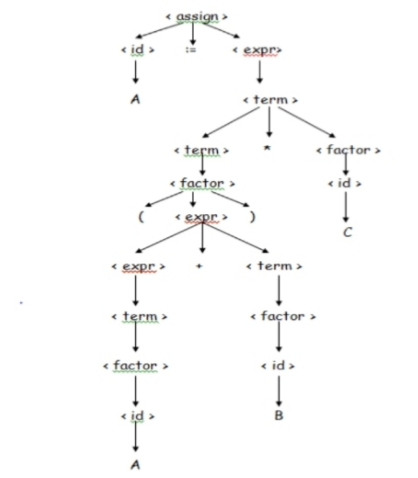
b. **A : = B + C + A**

c. **A : = A * (B + C)**

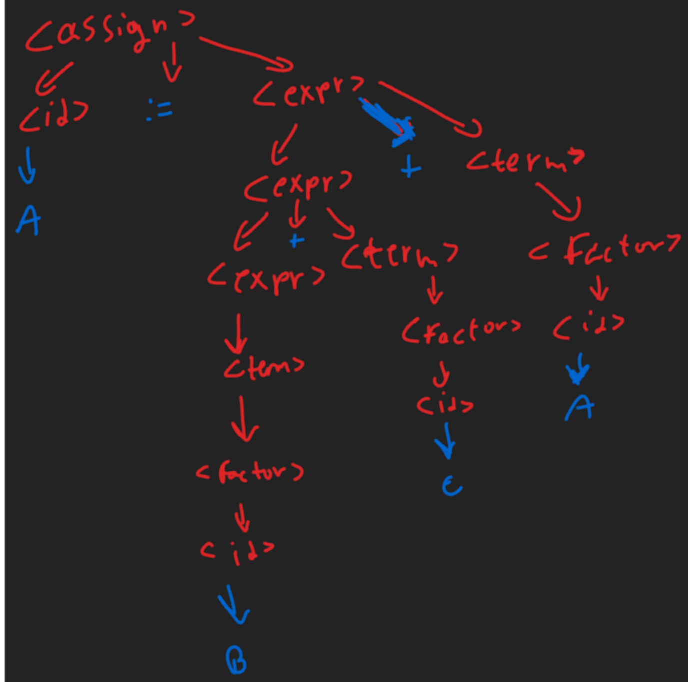
Solution

a.

< assign > → < id > := < expr >
→ A := < expr >
→ A := <term >
→ A := < term > * < factor >
→ A := < factor > * < factor >
→ A := (< expr >) * < factor >
→ A := (< expr > + < term >) * < factor >
→ A := (< term > + < term >) * < factor >
→ A := (< factor > + < term >) * < factor >
→ A := (< id > + < term >) * < factor >
→ A := (A + < term >) * < factor >
→ A := (A + < factor >) * < factor >
→ A := (A + < id >) * < factor >
→ A := (A + B) * < factor >
→ A := (A + B) * < id >
→ A := (A + B) * C .



② b) A := B + C + A



<assign> → <id> := <expr>
→ A := <expr>
→ A := <expr> + <term>
→ A := <expr> + <term> + <term>
→ A := <term> + <term> + <term>
→ A := <factor> + <term> + <term>
→ A := <id> + <term> + <term>
→ A := B + <term> + <term>
→ A := B + <factor> + <term>
→ A := B + <id> + <term>
→ A := B + C + <term>
→ A := B + C + <factor>
→ A := B + C + <id>
→ A := B + C + A